

Lesson 11 — Capstone and Extras

Node.js/Express

Game Plan

- Warm-Up
- What You've Built
- Enhancement Ideas
- Starting Your Own Project
- Team Workflow Basics
- Capstone Tips
- Wrap-Up

Warm-Up (5 min)

In chat or out loud:

1. What's the feature or concept you're most proud of from this course?
2. If you could add one thing to your app, what would it be?

Look What You've Built

Over 10 weeks, you implemented:

- A Node.js back end with Express
- REST API with authentication (JWT + cookies)
- PostgreSQL database with Prisma ORM
- Input validation (Joi) and password hashing
- Security middleware (helmet, rate limiting, XSS)
- CSRF protection
- Automated tests with Jest and Supertest
- Cloud deployment (Neon + Render)

That's a production-grade API. Seriously.

The Capstone Requirement

Your final project needs **something extra**.

This doesn't mean "do everything" — pick **one or two** ideas:

- OAuth with Google sign-in
- Todos organized into folders
- Role-based access control (admin vs. user)
- API documentation with Swagger
- Bulk update or delete of tasks
- A recycle bin (soft delete)
- A progress log for tasks

All changes go in a `lesson11` branch.

Enhancement: Folders

Extend the data model — add a `folders` table:

```
model Folder {  
  id      Int      @id @default(autoincrement())  
  name    String  
  userId  Int  
  Task    Task[]  
}
```

Then add:

- `POST /api/folders` — create a folder
- `GET /api/folders` — list folders
- Optional `folder` param on task create/update

Enhancement: Role-Based Access Control

Add a `role` column to users. Give certain users the `"manager"` role.

```
// In auth middleware
req.user = { id: payload.userId, role: payload.role };
```

```
// In a protected analytics route
if (req.user.role !== "manager") {
  return res.status(403).json({ message: "Forbidden." });
}
```

Managers can see all users' analytics. Regular users can't.

Enhancement: Swagger API Docs

Document your API so other developers can use it:

```
// Add comments above each route handler
/**
 * @swagger
 * /api/tasks:
 *   get:
 *     summary: Get all tasks for the logged-in user
 *     responses:
 *       200:
 *         description: List of tasks
 */
```

Generates a UI at `/api-docs` where anyone can browse and test your API.

Enhancement: Soft Delete (Recycle Bin)

Instead of actually deleting tasks, mark them as trashed:

```
model Task {  
  // ...existing fields  
  isTrashed Boolean @default(false) @map("is_trashed")  
}
```

- `DELETE /api/tasks/:id` → set `isTrashed: true`
- `GET /api/tasks` → exclude trashed tasks by default
- `DELETE /api/tasks/empty-trash` → actually delete trashed tasks

Bonus: support a `?includeTrash=true` query param.

How to Start Your Own Project

When you're ready to build something from scratch:

1. Create a GitHub repo (with README, license)
2. `git clone` it locally
3. `npm init` → set up `package.json`
4. Install your dependencies
5. Create `.gitignore` (include `node_modules/` , `.env`)
6. Branch protection on `main` — require PRs
7. Build, commit, push, create PRs

Don't skip the branch protection — it builds good habits.

Team Workflow Basics

On a real team:

- Work on **feature branches**, not `main`
- Submit **pull requests** — someone reviews before merging
- The `dev` branch is the active integration branch
- The `main` branch is production

This is the standard **trunk-based development** flow used in most companies.

Pull Request Culture

A PR is not just code — it's a conversation:

- Write a clear description of what you changed and why
- Reviewers check for correctness, security, style
- Address feedback before merging
- Never force-push to shared branches

If you want to stand out to employers: your GitHub history and PR descriptions matter.

We Do — Design Your Enhancement

Choose one enhancement idea and sketch it out:

1. What change to the schema (if any)?
2. What new routes or route changes?
3. What authorization rules apply?
4. How would you test it?

Take 5 minutes to discuss in pairs or small groups.

You Do (5 min)

Open the final project rubric.

Find the "something extra" section. Identify:

1. Which enhancement aligns with what you've already built?
2. What's the first code change you'd make?

Come back ready to share your plan.

Capstone Tips

- **Keep it back end.** Don't get pulled into React changes.
- **Test with Postman.** You don't need a front end for most enhancements.
- **Start simple.** One new route + one schema change is enough.
- **Protect every new route.** Any new route that returns user data needs auth.
- **Don't try to do everything.** One solid enhancement beats three half-finished ones.

Wrap-Up

Let's go around and each share:

1. What enhancement are you planning for your capstone?
2. What's your biggest takeaway from this course?
3. What concept do you still want to learn more about?

Resources for What's Next

- <https://www.prisma.io/docs> (keep exploring Prisma features)
- <https://expressjs.com/en/advanced/best-practice-security.html>
- <https://swagger.io/docs/specification/about/> (OpenAPI/Swagger)
- <https://owasp.org/www-community/attacks/> (security deep dive)
- Ask questions in Slack — even after the course

You Did It

You went from `console.log("Hello World")` in Node...

...to a deployed, authenticated, database-backed REST API.

That's real back end development.

Whatever comes next — practicum, job search, your own project — you have the foundation.

Closing

This course:

11 weeks. Node, Express, SQL, Prisma, Auth, Testing, Deployment.

What's next:

Practicum, your own projects, and everything you haven't learned yet.

Keep building. Good luck.